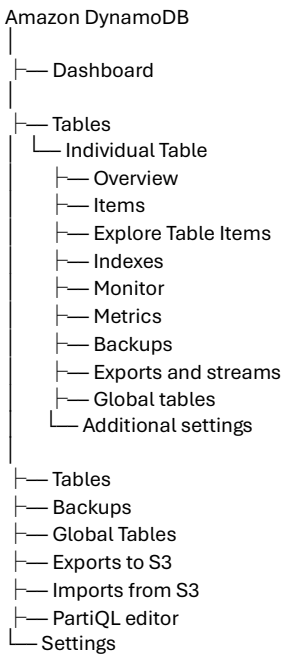
Amazon DynamoDB — Console Dashboard Wise Important Interview Questions & Answers



1. What is Amazon DynamoDB?

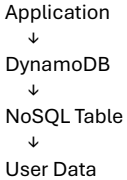
Answer:

Amazon DynamoDB is a fully managed NoSQL database service provided by AWS.
It is designed to provide fast and predictable performance at large scale.
AWS manages the servers, infrastructure, patching, and database availability.

For example:

Suppose we are building an application that stores millions of user sessions.
Instead of managing database servers ourselves, we can store the session data in DynamoDB.
DynamoDB can automatically handle large amounts of requests.

Example:



Simple ga gurthupettuko:

DynamoDB = Managed NoSQL + Fast + Highly Scalable

---

2. What is a DynamoDB Table?

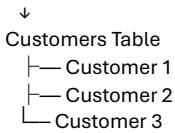
Answer:

A DynamoDB table is a collection of related items stored in DynamoDB.
It is similar to a table in a relational database, but its data model is NoSQL.
A table is created in an AWS Region and can automatically scale based on the selected capacity mode.

For example:

Suppose we have an application with customer information.
We can create a Customers table and store each customer's information as an item.
Each item can contain different attributes.

Example:



Simple ga gurthupettuko:

Table = Collection of DynamoDB items

---

3. What is an Item in DynamoDB?

Answer:

An item is an individual record stored inside a DynamoDB table.
It is similar to a row in a relational database.
Each item contains one or more attributes.

For example:
Suppose our Users table contains customer information.
One customer such as Ravi can be stored as one item.
That item can contain attributes such as name, email, and age.
Example:
Users Table
↓
One Item
├— UserId
├— Name
└— Email
Simple ga gurthupettuko:
Item = One record in DynamoDB

---

4. What is an Attribute in DynamoDB?
Answer:
An attribute is a piece of data inside a DynamoDB item.
It is similar to a column value in a relational database.
DynamoDB items do not necessarily need to have exactly the same attributes.
For example:
One user item may have Name, Email, and Age.
Another user item may have Name, Email, and Phone.
DynamoDB allows this flexible data structure.
Example:
Item
├— Name = Ravi
├— Email = ravi@example.com
└— Age = 25
Simple ga gurthupettuko:
Attribute = Data field inside an item

---

5. What is a Primary Key in DynamoDB?
Answer:
A primary key uniquely identifies an item in a DynamoDB table.
DynamoDB supports two main primary-key designs: a partition key alone or a partition key combined with a sort key.
The primary key is very important because DynamoDB uses it to locate items efficiently.
For example:
Suppose we create a Users table with UserId as the partition key.
Every user must have a unique UserId.
DynamoDB uses that key to find the user's item.
Example:
Users Table
↓
Primary Key
↓
UserId
↓
User-101
User-102
User-103
Simple ga gurthupettuko:
Primary Key = Uniquely identify an item

---

6. What is a Partition Key?
Answer:
A partition key is the key DynamoDB uses to determine where an item is stored.
It is also called a hash key.
DynamoDB uses the partition-key value to distribute data across its underlying infrastructure.
For example:
Suppose UserId is the partition key.
When we store users with different UserId values, DynamoDB uses those values to distribute the data.
This helps DynamoDB scale efficiently.
Example:
UserId
↓
DynamoDB
↓
Partitioning / Distribution
↓
Stored Data

Simple ga gurthupettuko:
Partition Key = Determines data distribution

---

7. What is a Sort Key?
Answer:
A sort key is used together with a partition key to create a composite primary key.
Multiple items can have the same partition-key value as long as their sort-key values are different.
It also allows related items to be stored and queried together.
For example:
Suppose UserId is the partition key and OrderId is the sort key.
One user can then have multiple orders with different OrderId values.
We can query all orders belonging to a particular user.
Example:
UserId + OrderId
↓
Composite Primary Key

User-101 + Order-1
User-101 + Order-2
User-101 + Order-3
Simple ga gurthupettuko:
Sort Key = Organize multiple items under same partition key

---

8. What is the difference between Partition Key and Sort Key?
Answer:
The partition key determines the logical partition for an item.
The sort key is used with the partition key to uniquely identify items within that partition.
A table can use only a partition key or both a partition key and sort key.
For example:
Suppose CustomerId is the partition key and OrderDate is the sort key.
A customer can have multiple orders with different dates.
This lets us query that customer's orders efficiently.
Example:
CustomerId = C101
+
OrderDate
↓
C101 + Jan
C101 + Feb
C101 + Mar
Simple ga gurthupettuko:
Partition Key = Where
Sort Key = Order within that partition

---

9. What is Capacity Mode in DynamoDB?
Answer:
Capacity Mode determines how DynamoDB handles read and write capacity for a table.
The two main options are **Provisioned** and **On-Demand**.
We choose the mode based on whether the workload is predictable or unpredictable.
For example:
Suppose our application has a predictable workload every day.
We can use Provisioned capacity and configure the required read and write capacity.
For unpredictable traffic, On-Demand can be a better choice.
Example:
DynamoDB Table
↓
Capacity Mode
┌──────────┐
↓ ↓
Provisioned On-Demand
Simple ga gurthupettuko:
Capacity Mode = How DynamoDB handles traffic capacity

---

10. What is DynamoDB On-Demand Capacity Mode?
Answer:
On-Demand mode allows DynamoDB to automatically handle read and write traffic without requiring us to provision capacity in advance.
It is useful for unpredictable or rapidly changing workloads.
We pay based on the requests processed by the table.
For example:
Suppose an application normally receives low traffic but suddenly gets a large number of requests.

With On-Demand mode, DynamoDB automatically handles the changing request volume.
We do not need to manually calculate the initial capacity.
Example:
Traffic
↓
Low → High → Low
↓
DynamoDB On-Demand
↓
Automatically Handles Requests
Simple ga gurthupettuko:
On-Demand = No capacity planning

---

11. What is Provisioned Capacity Mode?
Answer:
Provisioned mode allows us to specify the read and write capacity for a DynamoDB table.
It is useful when the application's workload is predictable.
We can also configure Auto Scaling to adjust provisioned capacity based on demand.
For example:
Suppose our application consistently needs around a certain level of read and write traffic.
We can provision the required capacity for the table.
If traffic changes, DynamoDB Auto Scaling can adjust the capacity when configured.
Example:
DynamoDB
↓
Provisioned Capacity
├— Read Capacity
└— Write Capacity
Simple ga gurthupettuko:
Provisioned = Plan capacity beforehand

---

12. What is a DynamoDB Index?
Answer:
An index provides another way to query data in a DynamoDB table.
Instead of querying only by the table's primary key, we can create indexes using different key attributes.
The two main index types are GSI and LSI.
For example:
Suppose our table uses UserId as its primary key, but we frequently search users by Email.
We can create an index using Email so that the application can query users efficiently using that attribute.
Example:
DynamoDB Table
↓
Index
↓
Query using another key
Simple ga gurthupettuko:
Index = Another efficient way to query data

---

13. What is a Global Secondary Index (GSI)?
Answer:
A Global Secondary Index allows us to query a DynamoDB table using a different partition key from the table's primary key.
A GSI can have its own partition key and optional sort key.
It is commonly used when the application needs another access pattern.
For example:
Suppose our table's primary key is UserId, but we need to search users by Email.
We can create a GSI with Email as its partition key.
The application can then query the GSI using Email.
Example:
Users Table
Primary Key → UserId

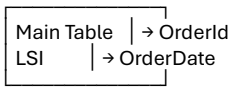
↓

GSI
Partition Key → Email

↓
Query by Email
Simple ga gurthupettuko:
GSI = Query using another key

---

14. What is a Local Secondary Index (LSI)?
Answer:
A Local Secondary Index uses the same partition key as the base table but can have a different sort key.
It must be created when the table is created.
It is useful when we need different sorting or querying options within the same partition.
For example:
Suppose CustomerId is the partition key and OrderId is the table's sort key.
We can create an LSI using OrderDate as another sort key.
Then we can query a customer's orders using that alternate sort order.
Example:
CustomerId
↓



Simple ga gurthupettuko:
LSI = Same Partition Key + Different Sort Key

---

15. What is DynamoDB Point-in-Time Recovery (PITR)?
Answer:
Point-in-Time Recovery protects DynamoDB table data by continuously backing it up.
It allows us to restore a table to a specific point in time within the supported recovery window.
It is useful for recovering from accidental deletion or unwanted changes.
For example:
Suppose someone accidentally deletes important items from a production table.
If PITR was enabled, we can restore the table to a point before the accidental deletion.
This helps recover important application data.
Example:
DynamoDB Table
↓
PITR
↓
Continuous Backup
↓
Restore to Required Time
Simple ga gurthupettuko:
PITR = Restore DynamoDB to an earlier time

---

16. What are DynamoDB Backups?
Answer:
DynamoDB provides backup capabilities to protect table data.
The console provides options for point-in-time recovery and on-demand backups.
Backups can be used to restore table data when required.
For example:
Suppose we are making a major application change that affects the database.
We can create an on-demand backup before the change.
If required, we can use the backup to restore the table.
Example:
DynamoDB Table
↓
Backup
↓
Stored Recovery Copy
↓
Restore
Simple ga gurthupettuko:
Backup = Protect DynamoDB table data

---

17. What is a DynamoDB Global Table?
Answer:
A Global Table allows a DynamoDB table to be replicated across multiple AWS Regions.
It is designed for globally distributed applications that need low-latency access and regional resilience.
Applications can access data from a nearby Region.
For example:
Suppose our application has users in India and the US.
We can create a Global Table with replicas in both Regions.
Users can access the database from a nearby Region.
Example:
India Region
↓

DynamoDB Replica
↕
Global Table
↕
DynamoDB Replica
↓
US Region
Simple ga gurthupettuko:
Global Table = DynamoDB across multiple Regions

---

18. What is DynamoDB Streams?
Answer:
DynamoDB Streams captures item-level changes made to a DynamoDB table.
It can record events such as item creation, update, and deletion.
Applications can use these changes for event-driven processing.
For example:
Suppose a new order is added to a DynamoDB table.
DynamoDB Streams can capture that change, and another AWS service such as Lambda can process the event.
This can be used for notifications, auditing, or additional processing.
Example:
Application
↓
DynamoDB
↓
DynamoDB Streams
↓
Lambda
↓
Process Change
Simple ga gurthupettuko:
Streams = Capture table changes

---

19. What is Export to S3 in DynamoDB?
Answer:
DynamoDB can export table data to Amazon S3.
This allows us to use the data for analytics, reporting, backups, or other processing without directly impacting normal application access.
The exported data can then be processed using other AWS services.
For example:
Suppose our application stores millions of orders in DynamoDB.
We can export the table data to S3 for analytics.
Services such as Athena can then be used to analyze the exported data.
Example:
DynamoDB
↓
Export
↓
Amazon S3
↓
Analytics
Simple ga gurthupettuko:
Export to S3 = Move DynamoDB data to S3 for analysis

---

20. What is PartiQL in DynamoDB?
Answer:
PartiQL is a SQL-compatible query language supported by DynamoDB.
It allows us to work with DynamoDB data using SQL-like statements.
It is available through the DynamoDB console's PartiQL editor and supported APIs.
For example:
Suppose we want to retrieve items from a DynamoDB table using a SQL-like syntax.
We can use the PartiQL editor in the AWS Console to run supported statements.
This can make data exploration easier for people familiar with SQL.
Example:
AWS Console
↓
PartiQL Editor
↓
SQL-like Query
↓
DynamoDB Table

Simple ga gurthupettuko:
PartiQL = SQL-like way to work with DynamoDB

---

21. What is the DynamoDB Monitor/Metrics section used for?
Answer:
The DynamoDB monitoring section helps us understand table activity and performance.
CloudWatch metrics can show information about requests, throttling, consumed capacity, and other table activity.
These metrics help us troubleshoot performance and capacity issues.
For example:
Suppose users are experiencing slow or failed requests.
We can check DynamoDB metrics to see whether the table is receiving high traffic or experiencing throttling.
Based on the metrics, we can adjust the table configuration.
Example:
DynamoDB
↓
CloudWatch Metrics
├— Requests
├— Capacity
└— Throttling
Simple ga gurthupettuko:
Metrics = Understand DynamoDB performance

---

22. What is DynamoDB TTL?
Answer:
DynamoDB Time to Live, or TTL, automatically deletes expired items from a table based on a configured expiration attribute.
It is useful for temporary data that should not remain in the database forever.
TTL helps reduce the need for applications to manually delete expired records.
For example:
Suppose we store user session data that should expire after one day.
We can store an expiration timestamp in a TTL attribute.
DynamoDB can automatically remove the expired items.
Example:
Session Item
↓
TTL = Expiration Time
↓
Expires
↓
DynamoDB Removes Item
Simple ga gurthupettuko:
TTL = Automatically remove expired items